



جامعة حلب
كلية الهندسة المعلوماتية
قسم هندسة البرمجيات ونظم المعلومات
السنة الخامسة

تطبيق لإدارة المنزلية الطارئة

DarCare

إعداد الطلاب:

عمرو كلاس، آية حمودة، إيناس سلخو
محمد حذيفة الأحمد، ليلى حداد، مهديّة ماياتي

إشراف: د. مروة دحدوح

2025-2026

يغطي هذا المشروع أحد متطلبات نيل الإجازة في الهندسة المعلوماتية

ملخص

يهدف هذا البحث إلى تصميم وتنفيذ منصة لإدارة خدمات الطوارئ المنزلية بمرونة عالية وقابلية للتوسع. ولضمان استقرارية النظام وجودة بنائه، حيث انطلق بناء النظام من بنية أحادية مركبة (Modular Monolith) لضمان الفصل المنطقي الدقيق للعمليات والترابط الضعيف (Loose Coupling) بين أجزاء النظام. وقد شكل هذا التصميم أساساً متيناً مهّد الطريق للانتقال بسلاسة ونجاح إلى بنية الخدمات المصغرة (Microservices Architecture). تُرجم ذلك عملياً بفصل النظام إلى خدمتين مستقلتين: الخدمة الرئيسية (Main) وخدمة المواقع (Location) يتيح هذا النهج تحسين قابلية الصيانة، وعزل الأعطال، مع الحفاظ على كفاءة الأداء وتنظيم إدارة الخدمات داخل المنصة.

الفهرس

1.....	الفصل الأول: مقدمة
3.....	الفصل الثاني: تعريف المشروع
4.....	1-2 مقدمة
4.....	2-2 الهدف من المشروع
4.....	3-2 أهداف المشروع التفصيلية
4.....	4-2 الهدف الفرعي (الرقمي)
5.....	5-2 توصيف المشروع
5.....	6-2 آلية عمل المنصة
5.....	7-2 مميزات النظام
5.....	8-2 الجانب التقني
6.....	9-2 خدمات التي تقدمها المشروع
8.....	الفصل الثالث: التقنيات البرمجية المستخدمة
9.....	1-3 مقدمة
9.....	2-3 لغة HTML
10.....	3-3 لغة HTML5
10.....	4-3 لغة CSS
11.....	5-3 لغة JavaScript
12.....	6-3 لغة TypeScript
12.....	7-3 إطار العمل React
13.....	8-3 إطار العمل Material UI
13.....	9-3 بيئة التشغيل Laravel
14.....	10-3 إطار العمل Flutter
14.....	1-10-3 ميزات Flutter

15.....	2-10-3 Dart لغة
15.....	11-3 قاعدة البيانات MySQL
16.....	12-3 أداة GIT
17.....	13-3 أداة خدمات إضافية
18.....	الفصل الرابع : تحليل المشروع
19.....	1-4 مقدمة
19.....	2-4 نموذج الكيانات والعلاقات (ERD)
28.....	الفصل الخامس : واجهات المشروع
29.....	1-5 مقدمة
29.....	2-5 إدارة الحسابات والمصادقة (Authentication)
31	3-5 واجهات (User Module)
34.....	4-5 واجهات مقدم الخدمة/الحرفي (Provider Module)
36.....	5-5 الخدمات المشتركة والتواصل (Shared services)
38.....	6-5 لوحة تحكم الأدمن (Admin panel)
41.....	الفصل السادس : الإستنتاجات والمقترحات
42.....	1-6 الاستنتاجات
42.....	2-6 المقترحات
43.....	3-6 التحديات
44.....	المراجع:
45.....	الشكر والتقدير:
46.....	:Abstract

.....

الفصل الأول

مقدمة

في عصر التطبيقات الرقمية، أصبح الاعتماد على الحلول السريعة والذكية ضرورة لا غنى عنها لمواجهة التحديات اليومية، خاصة في الحالات الطارئة داخل المنازل. ومع ذلك، لا تزال عملية الوصول إلى خدمات الطوارئ المنزلية (كهرباء، سباكة، تكييف، وغيرها) تعاني من العشوائية، وصعوبة العثور على مزود خدمة موثوق خلال وقت قصير، مما قد يؤدي إلى تأخير الحلول وزيادة حجم المشكلة.

من هنا، تنبع فكرة إنشاء منصة إلكترونية متخصصة تهدف إلى تنظيم وربط العملاء بمقدمي الخدمات المحليين (الفنيين) بطريقة مباشرة وآمنة، حيث تتيح للمستخدمين طلب الخدمة بسهولة لتصلهم المساعدة في أسرع وقت ممكن. تعتمد المنصة على نظام رقمي منظم يضمن سرعة الاستجابة وجودة الخدمة، مع توفير بيئة موثوقة تقلل من مخاطر الاحتيال وتضمن حقوق الطرفين.

ولتحقيق كفاءة عالية في التطوير وصيانة النظام، تم تبني استراتيجية تصميم تدرجية. بدأ العمل بتأسيس المنصة وفق بنية أحادية مركبة (Modular Monolith)، وهو ما أسهم في تحديد حدود واضحة (Bounded Contexts) لكل وحدة برمجية وتقليل الاعتماديات المتشابكة. هذا التصميم المنظم جعل من عملية الانتقال اللاحقة إلى بنية الخدمات المصغرة (Microservices Architecture) عملية سلسة ومدرسة هندسياً. فقد تم استخراج الوحدات وفصلها إلى مشاريع مستقلة، وتحديداً مشروع الخدمة الرئيسية (Main) ومشروع خدمة المواقع (Location) تتواصل هذه الخدمات فيما بينها عبر الشبكة، مما يوفر للمنصة مرونة عالية وقابلية للتوسع المستقبلي دون تعقيدات، ويقدم حلاً عملياً ومتكاملاً يرتقي بتجربة التعامل مع خدمات الطوارئ المنزلية.

الفصل الثاني

تعريف المشروع

2-1 مقدمة:

في عصر التطبيقات الرقمية، تظهر الحاجة المتزايدة إلى حلول فورية وموثوقة لمشكلات الطوارئ المنزلية (كهرباء، سباكة، تكييف، إلخ)، حيث يعاني الأفراد من صعوبة الوصول إلى فنيين موثوقين بسرعة، خاصة في الحالات العاجلة. من هنا، تأتي فكرة إنشاء منصة إلكترونية تربط بين العملاء ومقدمي الخدمات المحليين، بهدف تسهيل عملية طلب الخدمات بشكل سريع وآمن ومنظم.

2-2 الهدف من المشروع:

يهدف المشروع إلى إنشاء منصة إلكترونية متخصصة في خدمات الطوارئ المنزلية، تعمل على ربط العملاء بمقدمي الخدمات (الفنيين) بطريقة سريعة وآمنة ومنظمة، بما يساهم في تحسين تجربة الوصول إلى هذه الخدمات وتقليل الوقت والجهد المبذول في البحث عنها.

2-3 أهداف المشروع التفصيلية :

توفير وسيلة سهلة وسريعة لطلب خدمات الطوارئ المنزلية في أي وقت، ضمان بيئة آمنة وموثوقة تقلل من مخاطر الاحتيال وسوء الخدمة، تحسين جودة الخدمات من خلال نظام تقييم ومنافسة بين الفنيين، دعم الفنيين عبر توفير فرص عمل منظمة ومستقرة، تقليل الاعتماد على الطرق التقليدية والعشوائية في البحث عن الخدمات، رفع كفاءة الاستجابة للحالات الطارئة داخل المنازل.

2-4 الهدف الفرعي (الرقمي) :

بناء نظام رقمي متكامل يعتمد على تقنيات حديثة يساهم في تطوير قطاع الخدمات المنزلية، ويعزز من كفاءته واستدامته.

2-5 توصيف المشروع :

يقوم المشروع على تطوير منصة إلكترونية متخصصة في خدمات الطوارئ المنزلية، تهدف إلى ربط الأفراد الذين يواجهون أعطالاً مفاجئة (كهرباء، سباكة، تكييف، وغيرها) بمقدمي الخدمات (الفنيين) بطريقة سريعة وآمنة. تتيح المنصة للمستخدم تقديم طلب خدمة بسهولة، ليتم عرضه على مجموعة من الفنيين القريبين جغرافياً، مما يساهم في تسريع عملية الاستجابة وتحسين جودة الخدمة.

2-6 آلية عمل المنصة :

تعتمد المنصة في تسيير عملياتها على نمط الاتصال المباشر المتزامن Synchronous Direct Communication بين الخدمات المصغرة. حيث تتولى الخدمة الرئيسية (Main Project) إدارة سير العمليات الأساسية للمستخدمين، وعند الحاجة لمعالجة بيانات جغرافية (مثل البحث عن مزودي الخدمة ضمن نطاق محدد أو جلب إحداثيات مزود معين)، تقوم الخدمة الرئيسية بالتواصل مع خدمة المواقع (Location Project) عبر الشبكة باستخدام بروتوكول HTTP تم تجريد هذه العملية برمجياً من خلال بناء طبقة عميل مخصصة (Service Client) داخل الخدمة الرئيسية، والتي تقوم بإرسال الطلبات المتزامنة (RESTful Requests) ومعالجة الردود بشكل آمن، مما يضمن استقلالية خدمة المواقع في إدارة مخطط بياناتها الجغرافية مع الحفاظ على تكامل تدفق العمليات في المنصة.

2-7 مميزات النظام :

واجهة استخدام بسيطة وسهلة لجميع الفئات، سرعة الوصول إلى الفنيين في الحالات الطارئة، نظام تقييم يعزز من جودة وموثوقية الخدمات، تقليل مخاطر الاحتيال عبر بيئة منظمة وآمنة، إمكانية إضافة تفاصيل ومرفقات توضح طبيعة المشكلة.

2-8 الجانب التقني :

اعتمد النظام في مراحل بنائه الأولى على نمط (Modular Monolith) الذي ساعد في تنظيم الشيفرة البرمجية وفصل مجالات العمل منطقياً. وبفضل هذا الترابط الضعيف (Loose Coupling)، تم الانتقال بكفاءة إلى بنية الخدمات المصغرة (Microservices Architecture)، حيث تم تقسيم النظام فعلياً إلى مشاريع مستقلة مشروع Main ومشروع Location وقد برزت الفوائد التقنية لهذا النهج في النقاط الآتية:

1. الاستفادة من التنظيم المسبق: كان استخراج مشروع (Location) أمراً مباشراً بفضل الحدود الواضحة (Bounded Contexts) التي تم تأسيسها مسبقاً في البنية الأحادية المركبة.

2. التكامل عبر عملاء الخدمات (Service Clients) : تم ربط الخدمات باستخدام واجهات برمجة التطبيقات (REST APIs) ، حيث تم بناء صنف برمجي مخصص (LocationClient) في المشروع الرئيسي ليكون مسؤولاً عن تبادل البيانات الجغرافية بدقة وفعالية.
3. تطبيق مبادئ التسامح مع الأخط (Fault Tolerance) : تم تزويد الاتصال الشبكي بآليات التحكم بالوقت (Timeouts) ، مما يضمن عدم تجمد أو توقف الخدمة الرئيسية (Main) في حال تأخر استجابة خدمة المواقع (Location) ، محققاً بذلك عزلاً حقيقياً للأعطال Fault Isolation .
4. استقلالية النشر وقابلية التوسع : إمكانية بناء ونشر وتوسيع كل خدمة على حدة وبشكل فيزيائي مستقل لتلبية احتياجات النظام بكفاءة.

2-9 خدمات التي تقدمها المشروع :

1. خدمات مقدمة للعملاء :

طلب الخدمة مع تحديد نوعها وموقعها، متابعة حالة الطلب لحظة بلحظة، التواصل المباشر مع مقدم الخدمة، تقييم الخدمة بعد الانتهاء منها.

2. الوحدات المخصصة للحرفيين :

استقبال الطلبات ضمن نطاق جغرافي محدد. إدارة أنواع الخدمات المقدمة وأسعارها. متابعة الأرباح والمدفوعات.

3. وحدات الإدارة داخل المنصة :

إدارة حسابات المستخدمين (تجار، عملاء، مندوبين). مراقبة الأداء العام للنظام. التحكم بالإشعارات والتنبيهات.

4. الخدمات والمجالات الأساسية (Core Domains & Services) :

1. وحدة المستخدمين (User Module) : إدارة الحسابات وبيانات المصادقة (الهوية، الجلسات).
2. وحدة الإشعارات (Notification Module) : إرسال التنبيهات والتحديثات الفورية للمستخدمين.
3. الوحدة الأساسية (Core Module) : تضم العناصر والوظائف المشتركة بين باقي الوحدات.
4. وحدة الطلبات (Order Module) : إدارة دورة حياة الطلب (الإنشاء، التحديث، تتبع الحالة).

5. وحدة لوحة التحكم (Dashboard Module) : واجهة موحّدة لعرض البيانات والتقارير والتحكم بالنظام.

6. خدمة المواقع (Location Service) : خدمة مستقلة مسؤولة عن معالجة وتخزين الإحداثيات الجغرافية، وحساب المسافات، والبحث عن أقرب مزودي الخدمة للمستخدمين ضمن نطاق محدد.

الفصل الثالث

التقنيات البرمجية المستخدمة

3-1 مقدمة:

تتضمن هذه التقنيات اللغات البرمجية والإطارات البرمجية ومكتبات البرمجة والأدوات المساعدة وغيرها من الأدوات التي تمكن المطورين من إنشاء برامج عالية الجودة والأداء.

3-2 لغة HTML:

Hyper Text Markup Language ويشار إليها اختصاراً HTML تم وصفها في اواخر 1990 من قبل Tim Berners-Lee هي لغة تصميم صفحات ومواقع ويب، وتعتبر من أقدم اللغات و اوسعها استخداماً وهي ليست لغة برمجة ولا تحتاج الى مترجم خاص بها و غير مرتبطة بنظام تشغيل معين لأنه يتم تفسيرها وتنفيذ تعليماتها مباشرة من قبل المتصفح ، يمكننا القول بأنها عبارة عن الهيكل الرئيسي لصفحات الويب والبنية التحتية لها، إذ تقدم وصفاً مفصلاً حول الكيفية التي ستكون عليها آلية عرض محتويات الموقع الإلكتروني ويكون ذلك بتقسيمه إلى عنوان وفقرات ويتم ذلك كله بالاعتماد على ما يُعرف بالوسوم Tags ويقوم متصفح الويب بتحويل السطور المكتوبة بهذه اللغة إلى صفحات مفهومة وسهلة القراءة لزوار الموقع تتألف صفحات HTML من عناصر، والتي تتألف عادة من وسم بداية و وسم نهاية، ويكون بينهما محتوى نصي عادة، ويمكن أن نضع في وسم البداية بعض الخصائص التي تغير من سلوك العنصر او تؤدي الى ضبطه. لاحظ أن بعض العناصر يمكن ان تتشعب داخل بعضها بعضاً. الأقسام الأساسية للعناصر هي:

1. وسم البداية: وهو يحتوي على اسم العنصر، موضوعاً ضمن قوسين على شكل زاوية، وقد يلي الاسم الخاصيات التي تؤثر عليه. مثلاً <p>.
2. وسم النهاية: وهو يحتوي على اسم العنصر ايضاً مسبوق بخط مائل قبله للإشارة الى نهاية العنصر، ولاحظ أن نسيان وسم النهاية قد يسبب أخطاء في بعض الأحيان، لذا خذ حذرك وتذكره. مثلاً <p>.
3. المحتوى: وهو موجود بين وسمي البداية والنهاية، ويمثل في معظم الاحيان محتوى العنصر.
4. العنصر: هو وسم البداية و وسم النهاية اضافة الى المحتوى. [5]

3-3 لغة HTML5:

تم في هذا الإصدار إضافة العديد من العناصر والميزات الجديدة حيث تم بناء هذه المميزات اعتماداً على HTML DOSS, Java Script وتم إضافة ميزات أفضل في عملية معالجة الأخطاء.

المميزات المضافة في HTML5:

1. العنصر canvas لتنفيذ التعليمات الرسومية.
2. العنصرين video audio لتشغيل ملفات الصوت والصورة.
3. دعم أفضل لتخزين الصفحات في نمط الاتصال Offline Storage.
4. عناصر جديدة لدعم أنواع معينة من المحتوى مثل article, footer, header.
5. عناصر إدخال جديدة مثل calendar, date, time, email.

4-3 لغة CSS:

تعتبر CSS اختصار ل (Cascading Style Sheet) - لغة توصيفيه تقوم بمنح موقع الويب شكله الجميل وتصميمه الفريد الذي يميزه عن غيره من المواقع. تعد لغة ال CSS صديقة للغة ال HTML دائماً. فهي مرافقة لها وبجانبها في تصميم وإنشاء صفحات الويب. تشكل كل من هاتان اللغتان الأساس لكل موقع فال HTML هي أساس كل عناصر صفحة الويب أما ال CSS فهي أساس تصميم وشكل الموقع فبدونها ستظل مواقع الويب نصاً عادياً على خلفيات بيضاء، عليك تعلم CSS ان اردت ان تكون مطور واجهات متخصص، كما عليك بالتأكد تعلم HTML لبناء اي موقع متميز. قبل البدء ببناء وتطوير لغة ال CSS في عام 1996 من قبل اتحاد شبكة الويب العالمية (W3C)، كانت صفحات الويب محدودة وبسيطة للغاية من حيث الشكل والوظيفة. فقد كانت المتصفحات القديمة تعرض صفحات الويب كصفحات تتكون من نصوص سوداء بخلفية بيضاء فقط (نص - صور - روابط.. إلخ) بدون أي تصميم يذكر. ولم يكن هناك تخطيط لموعد إطلاق لغة CSS. [1]

سمحت CSS بتصميم وتشكيل صفحات الويب بطرق متعددة ومختلفة لتتمكن من استيعاب مدى إبداع الشخص مثل القدرة على:

1. تحديد الخطوط والهوامش والمسافات لعناصر الموقع
2. تحديد لون وحجم العناصر
3. تطبيق الألوان على الخلفيات
4. تحريك العناصر بحرية والحصول على العديد من الحركات في الموقع. والعديد من المزايا الأخرى.

3-5 لغة JavaScript:

جافا سكريبت أو في الغالب يطلق عليها اختصارًا JS بالإنجليزية JavaScript: هي لغة برمجة عالية المستوى تستخدم أساساً في متصفحات الويب لإنشاء صفحات أكثر تفاعلية يتم تطويرها حالياً من طرف شركة موزيلا.

كانت لغة الجافا سكريبت موجهة للمبرمجين الهواة وغير المحترفين، إلا أنه تزايد الاهتمام بها وجذبت اهتمام مبرمجين محترفين بعد إضافتها لتقنيات جديدة كانتشار تقنية أجاكس التي أدت إلى سرعة في التفاعل بين الخادم والعميل.

تُستخدم لغة الجافا سكريبت لإنشاء صفحات ويب تفاعلية، ولتوفير تطبيقات ويب بما في ذلك الألعاب؛ وهي مُستعملة من أغلبية المواقع، وتدعمها جميع المتصفحات تقريباً دون الحاجة إلى إضافات خارجية.

في الماضي كانت لغة JavaScript محدودةً إلى متصفحات الويب فيما قد سلف، لكن توسع مجال JavaScript في الفترة الأخيرة لتضم إنشاء تطبيقات ويب من جهة الخادم (server-side) مع التعامل مع قواعد البيانات، إضافة إلى إمكانية استخدامها في بيئات ليست متعلقة بالويب مثل معالجات النصوص وبرمجيات PDF، ومن الممكن استعمالها أيضاً لكتابة تطبيقات سطح المكتب أو تطبيقات الهواتف، وحتى تطوير الألعاب.

يتألف أساس لغة JavaScript من الميزات الشائعة للغات البرمجة، التي تسمح لنا:

1. بتخزين القيم داخل متغيرات
2. إجراء عمليات معالجة على السلاسل النصية
3. تنفيذ إجراءات معينة اعتماداً على وقوع الأحداث.

ما يجب أن يثير اهتمامنا هو الواجهات البرمجية التي تُضيف وظائف على لغة JavaScript، وتسمى هذه الواجهات البرمجية بالمصطلح APIs

(Application Programming Interfaces)، والتي تسمح للمطورين باستخدام لغة JavaScript للتعامل مع كثير من الأشياء، في مجال متصفحات الويب يمكننا استخدام JavaScript للتعامل مع شجرة DOM، ولإستخدام الواجهة البرمجية للمواقع الجغرافية (Geolocation API)، ولإنشاء رسومات ثنائية وثلاثية الأبعاد عبر Canvas و WebGL، وللتحكم بالوسائط مثل الفيديو والصوت، وغير ذلك. [6]

يمكن تضمين شيفرات JavaScript في الصفحات بطريقتين:

1. تضمين الشيفرات داخل مستند HTML.

2. استخدام ملف خارجي.

يمكن تضمين شيفرة JavaScript في أي مكان داخل المستند، ولكن يُنصح بوضعها قبل وسم الاغلاق للعنصر `<body>` تُضاف الشيفرة داخل العنصر `<script>` أما إذا كانت شيفرة JavaScript في ملف خارجي وليكن اسمه `script.js` فيمكن تضمينه باستخدام العنصر `<script>` أيضاً، لكن مع استخدام الخاصية `src`.

3-6 لغة TypeScript:

هي لغة برمجة تم إنشاؤها من قبل مايكروسوفت وهي تعتبر إضافة إلى JavaScript. تم تصميم TypeScript ليكون عبارة عن نظام كتابة الأكواد يساعد على تطوير تطبيقات ويب أكثر قوة وصحة. تتميز TypeScript بتوفير نظام قوي للأنواع يتيح للمطورين تحديد أنواع البيانات الخاصة بكل عنصر من عناصر البرنامج، مما يساعد على تحسين جودة الأكواد وتجنب الأخطاء الناتجة عن تحويل الأنواع. كما أن TypeScript يدعم العديد من الميزات الحديثة في JavaScript مثل الوعود (Promises) والتعبيرات الشرطية (Conditional Expressions) والتعبيرات العامة (Arrow Functions). تساعد TypeScript على تحسين إنتاجية المطورين وتسهيل عليهم القيام بالأعمال الخاصة بالتعامل مع الأكواد، وتجعل عملية التطوير أكثر سلاسة وسهولة وأمان. [10]

3-7 إطار العمل React:

هو إطار عمل (Framework) لتطوير واجهات المستخدم (UI)، وهو يستخدم لبناء تطبيقات الويب الديناميكية. تم تطوير ReactJS بواسطة فريق عمل في Facebook، وتم إصداره لأول مرة في عام 2013. ومنذ ذلك الحين، أصبح ReactJS شائعاً جداً ومشهوراً في مجتمع تطوير الويب. ما يميز ReactJS هو أسلوبه في إنشاء واجهات المستخدم الديناميكية. يعتمد React على مفهوم المكونات (Components)، حيث يتم تجزئة واجهة المستخدم إلى عناصر صغيرة وقابلة لإعادة الاستخدام تسمى المكونات. تتفاعل المكونات مع بعضها البعض لإنشاء واجهة مستخدم متجاوبة وديناميكية. يستخدم ReactJS أيضاً مفهوم "التركيز على الحالة" (State)، حيث يسمح للمطورين بتعريف حالة التطبيق وتتبع تغييراتها عبر الوقت. وعندما يتغير الحالة، يعيد React بناء الجزء المتأثر من واجهة المستخدم بدلاً من إعادة بناء الصفحة بأكملها، وهذا يؤدي إلى تحسين أداء التطبيق. يمكن استخدام ReactJS لبناء تطبيقات الويب الكبيرة والمعقدة، وهو متوافق مع العديد من التقنيات الأخرى وإطارات العمل (Frameworks)، مثل Redux و React Router. وبفضل مجتمعه النشط والدعم المستمر

من قبل Facebook والمطورين الآخرين، أصبح React واحدًا من الأدوات الأكثر شعبية واستخدامًا في مجال تطوير الويب. [8]

8-3 إطار العمل Material UI :

Material UI هي أداة تساعد في بناء واجهات المستخدم بشكل سريع وفعال. تعتمد على مفهوم التصميم بواسطة الأنماط، حيث يتم توفير مجموعة واسعة من الأنماط التي تقوم بتطبيق الأنماط والتنسيقات المختلفة على العناصر بدلاً من كتابة CSS مخصص.

من أهم ميزات Material UI :

1. سرعة التطوير: يمكنك بناء واجهات المستخدم بسرعة فائقة باستخدام الأنماط الموفرة مسبقًا دون الحاجة لكتابة CSS مخصص.
 2. قابلية التخصيص: يمكنك تخصيص التصميم بشكل كامل وفقًا لاحتياجات مشروعك، سواء عبر تعريف متغيرات جديدة أو إعادة تكوين قواعد الأسلوب المدمجة.
 3. تجربة مطور ممتعة: يتيح لك Material UI التركيز على بناء تجارب المستخدم بدلاً من القلق بشأن كتابة CSS، مما يجعل عملية التطوير أكثر متعة وإنتاجية.
- باختصار، Material UI توفر طريقة فعالة وسريعة لبناء واجهات المستخدم بطريقة مرنة وسهلة التخصيص، مما يجعلها خيارًا مثاليًا للمطورين الذين يرغبون في زيادة إنتاجيتهم وتسريع عملية التطوير. [14]

9-3 إطار العمل Laravel :

هو إطار عمل Framework مفتوح المصدر مبني باستخدام لغة PHP، ويُستخدم لتطوير تطبيقات الويب من جهة الخادم (Server-Side Web Development) بطريقة منظمة وسهلة. يعتمد Laravel على نمط التصميم MVC (Model - View - Controller)، والذي يساهم في فصل منطق التطبيق عن واجهة المستخدم، مما يجعل الكود أكثر ترتيبًا وسهولة في الصيانة. يقدم Laravel مجموعة من الأدوات والميزات التي تسهل عملية التطوير، من أهمها:

1. نظام التوجيه Routing لتنظيم الروابط داخل التطبيق.
2. التعامل مع قواعد البيانات باستخدام Eloquent ORM.
3. محرك القوالب Blade Template Engine لإنشاء الواجهات.
4. نظام المصادقة Authentication System لإدارة المستخدمين.
5. يُستخدم Laravel في تطوير العديد من تطبيقات الويب مثل:
6. المتاجر الإلكترونية E-commerce Websites.

7. أنظمة إدارة المحتوى (CMS (Content Management Systems)

8. لوحات التحكم Admin Dashboards

9. بناء واجهات برمجة التطبيقات (RESTful APIs) وتسهيل الاتصال الشبكي المتزامن بين

الخدمات المصغرة (Microservices) باستخدام أدوات مدمجة وفعالة مثل Laravel HTTP Client.

يعمل Laravel على مختلف أنظمة التشغيل مثل Linux و Windows و macOS، ويوفر بيئة تطوير قوية تساعد على بناء تطبيقات سريعة، آمنة، وقابلة للتوسع (Scalable Applications) [11].

10-3 إطار العمل Flutter:

هو إطار عمل من شركة Google مخصص لبرمجة تطبيقات الهواتف الذكية العاملة بنظام الاندرويد وال IOS وفوشيا (نظام تشغيل جديد من جوجل) حيث يركز على تجربة المستخدم و المطور في نفس الوقت و هذا لأنه يوفر إطار عمل شامل بلغة Dart مخصص لرسم الواجهات بجودة عالية و تطبيقات أصلية بأداء ممتاز و في نفس الوقت يعطي للمطور الأدوات التي تجعله يبني تطبيقات كاملة في أقل وقت. من مميزات Flutter التي جعلته أكثر سهولة لتطوير تطبيقات الهواتف هو إعماده على نظام (Widgets) فكل عنصر من Flutter يعتبر Widget وقد تم إستنباط هذا النظام من React JSX، وهذه تعتبر ميزة للمطورين اللذين سبق لهم التعامل مع React، فقد يجدون أن صناعة التطبيقات بإستخدام Flutter أكثر سهولة. نظام Widgets في إطار Flutter جعل من السهل إنشاء مكونات (Components) جديدة وقوة لغة Dart ساهمت أيضا بكونها لغة كائنية التوجه في هذا الأمر بحيث يمكن أخذ Widget رئيسي و إضافة عليه خصائص جديدة من تأثيرات و حركات (Animation) بشكل بسيط وبدون تعقيد. هذا الأمر مكن من إنشاء مكونات جديدة داخل الإطار و إدراج Material Design في المكونات التي تعمل على الأندرويد و Cupertino على IOS. [3]

10-3-1 مميزات Flutter:

هنالك عدة افكار جديدة جاء بها Flutter، حيث أن فلسفة فريق فلاتر هي امكانية رسم كل بكسل على الشاشة بحرية وعدم الارتباط بالنظام أي مثل الألعاب تماما و لكن العامل الذي جعل Flutter يصل إلى ما وصل إليه الآن هو دعم مباشر من فريق آخر في جوجل و هو فريق الماتريال ديزاين لان Flutter يوفر كل مكونات الماتريال ديزاين باصداراتها الاخيرة حتى قبل ان يحصل عليها الاندرويد بنفسه و هذا ما تم التصريح به في مؤتمر جوجل و من الاشياء التي يوجد فيها اختلاف كبير هو طريقة Flutter في رسم الواجهات لان Flutter يستخدم لغة البرمجة Dart في كل شيء من أجل الكتابة و التصميم ايضا.

3-10-2 لغة Dart:

هي لغة برمجة كائنية التوجه ظهرت في سنة 2011 وتم عملها من طرف شركة جوجل كمحاولة لاستبدال لغة جافا سكربت لكن لم تنجح وهذا واضح جدا لكن مع ذلك فهي توفر الخصائص المناسبة التي يحتاجها Flutter لكي ينمو بشكل صحيح

حيث ان لغة Dart هي:

1. لغة برمجة كائنية التوجه وتنفذ كل مفاهيم البرمجة الكائنية منها تعدد الوراثة تحت اسم mixin.
2. يتحكم فريق Flutter بشكل كامل في لغة Dart لكي تناسب Flutter.
3. اثناء البرمجة تستخدم لغة Dart مترجم JIT: Just in time و التي بفضلها ظهرت خاصية الهوت ريلود و هي انه عند تغير الكود يتم ارسال فقط البايتات التي تم تغييرها بالتالي حصل التغير في برنامجك في اقل من ثانية و يبقى محافظا على حالته .
4. اثناء نشر التطبيق تستخدم Dart مترجم AOT: Ahead of time و الذي ينتج لنا تطبيق اصلي و بالتالي اداء رائع .
5. يمكن ترجمة لغة Dart الى جافا سكربت محسنة الاداء وهذا ما يجعل عمل تطبيقات ويب ب Flutter امرا ممكنا .

6. لغة Dart تم عملها لكي تكون سهلة لمبرمج الجافا سكربت ومبرمجي الجافا و السي شارب أيضا. [2]

3-11 نظام قاعدة البيانات MySQL :

MySQL هو نظام إدارة قواعد بيانات علائقية (RDBMS) مفتوح المصدر وقوي للغاية، ويُعد من أكثر قواعد البيانات استخداماً في تطبيقات الويب. هنا بعض ميزاته الرئيسية:

1. أداء عالٍ وسرعة استجابة : يتميز MySQL بقدرته على التعامل مع قواعد البيانات الضخمة ومعالجة الاستعلامات المعقدة بسرعة، مما يجعله مثالياً لتطبيقات الويب ذات الحركة الكثيفة.
2. توافق ممتاز مع Laravel : يمتلك MySQL تكاملاً سلساً وطبيعياً مع إطار عمل Laravel ، مما يسهل عمليات التهجير (Migrations) وإدارة البيانات عبر Eloquent ORM .
3. أمان وموثوقية : يوفر MySQL ميزات أمان متقدمة تشمل تشفير البيانات، إدارة صارمة لصلاحيات المستخدمين، وآليات للنسخ الاحتياطي لضمان سلامة البيانات من فقدان أو الاختراق.

4. مرونة وقابلية للتوسع : يدعم النظام التوسع بشكل فعال ليتناسب مع نمو حجم البيانات، وهو متوافق تماماً مع بنية الخدمات المصغرة (Microservices) حيث يمكن تخصيص قاعدة بيانات مستقلة لكل خدمة.

5. مجتمع دعم ضخم : يحظى MySQL بشعبية واسعة ومجتمع نشط من المطورين، مما يعني وفرة كبيرة في المراجع، وحلولاً سريعة لأي تحديات برمجية قد تظهر أثناء التطوير.

12-3 أداة Git:

هو نظام تحكم في الإصدارات (Version Control System) يستخدم لمتابعة التغييرات على ملفات الكود والمشاريع البرمجية. يتيح Git للمستخدمين حفظ وتحديث الملفات والمشاريع بطريقة منظمة ومتسقة وذلك بحفظ نسخة من الملفات في كل مرة يتم فيها التعديل عليها.

ويسمح Git بتوزيع المشاريع البرمجية بين فريق عمل متعدد الأفراد والعمل عليها بشكل فعال ومنظم، حيث يمكن للمستخدمين العمل على نسخ مختلفة من المشروع بشكل مستقل ودمج التغييرات بسهولة فيما بعد. يستخدم Git بشكل واسع في عالم تطوير البرمجيات، ويدعم العديد من الخصائص المفيدة مثل الفروع (branches) والدمج (merging) والعلامات (tags) والتعليقات (commits) والتفرعات (forks) والمزيد. بالإضافة إلى ما ذكر سابقاً، يتميز Git بعدة ميزات وخصائص أخرى، مثل:

1. سرعة الأداء: يتميز Git بسرعه الفائقة في تنفيذ العمليات وإجراء التحديثات والاستعلامات، وهذا يعني أن المستخدمين يمكنهم العمل بشكل أسرع وأكثر كفاءة.
2. السهولة في التعلم والاستخدام: يعد Git سهل الاستخدام ويمكن تعلمه بسرعة، كما يوفر العديد من الأدوات والوظائف التي تسهل على المستخدمين العمل به.
3. القابلية للتخصيص: يمكن للمستخدمين تخصيص Git وفقاً لاحتياجاتهم الخاصة، بما في ذلك إضافة المزيد من الوظائف والأدوات التي يحتاجون إليها.
4. دعم العمل الجماعي: يتيح Git العمل الجماعي بكفاءة، حيث يمكن للمستخدمين العمل على المشروع بشكل مستقل ودمج التحديثات بسهولة.
5. الاستجابة للتغييرات: يتيح Git التعامل مع التغييرات بشكل فعال، حيث يمكن للمستخدمين إجراء تحديثات وإضافة ميزات جديدة بسرعة وسهولة.
6. الأمان: يقوم Git بحفظ ملفات المشروع بطريقة آمنة ومحمية، كما يمنع الوصول غير المصرح به إلى الملفات والتغييرات.

7. الدعم المتعدد للمنصات: يمكن استخدام Git على مختلف المنصات والأنظمة الخاصة بالحاسوب،

بما في ذلك Windows و Mac و Linux. [4]

13-3 خدمات إضافية :

خرائط جوجل (Google Maps) لتحديد المواقع و (Firebase) للأشعارات .

الفصل الرابع

تحليل المشروع

1-4 مقدمة :

يتضمن هذا الفصل تحليل قاعدة البيانات الخاصة بمنصة DarCare، حيث تم تصميمها وفق النموذج العلائقي (Relational Database Model) لضمان تنظيم البيانات وتحقيق التكامل المرجعي بين مختلف مكونات النظام. ويعتمد التصميم على مخطط ERD (Entity Relationship Diagram) الذي يوضح الكيانات الأساسية والعلاقات فيما بينها، مما يساهم في بناء قاعدة بيانات مرنة وقابلة للتوسع وسهلة الصيانة.

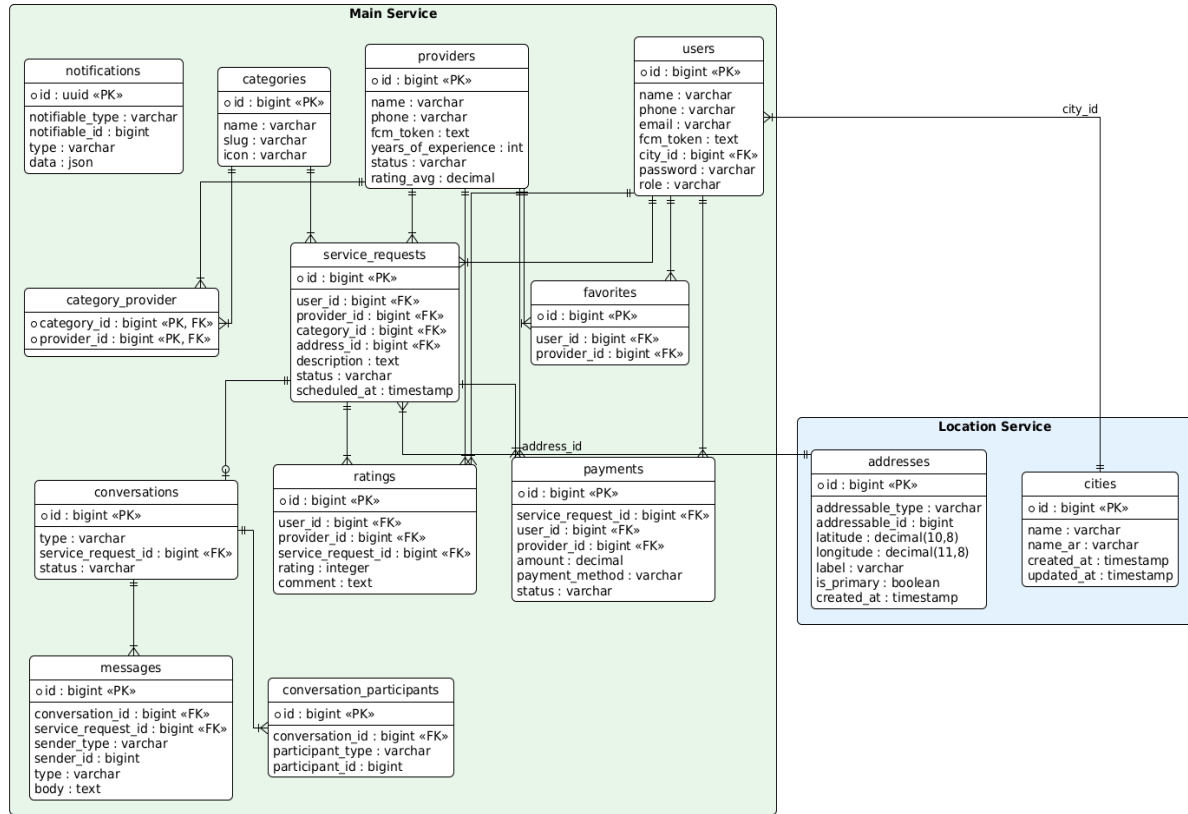
تضم قاعدة البيانات مجموعة من الجداول التي تمثل المستخدمين، والعملاء، ومقدمي الخدمات، والطلبات، والخدمات، والأقسام، والتصنيفات، والعناوين، وعمليات الدفع، والتقييمات، والرسائل، والإشعارات. وقد رُبطت هذه الجداول باستخدام المفاتيح الأساسية (Primary Keys) والمفاتيح الخارجية (Foreign Keys) بما يضمن سلامة البيانات ويمنع تكرارها، مع دعم العمليات المختلفة التي تنفذها المنصة بكفاءة وموثوقية.

2-4 نموذج الكيانات والعلاقات (ERD) :

يُعتبر مخطط ERD أحد وسائل تصميم قواعد البيانات. يُستخدم ERD لتصوير العناصر المختلفة في قاعدة البيانات والعلاقات بينها بشكل بصري. هذا يساعد في فهم وتصميم هيكل البيانات بشكل أفضل قبل تنفيذ قاعدة البيانات الفعلية.

يمكن أن يتضمن مخطط ERD ما يلي:

1. الكيانات (Entities): تمثل الكائنات المختلفة التي تحتوي على بيانات في قاعدة البيانات، مثل الجداول في قاعدة البيانات العلائقية.
2. العلاقات (Relationships): تصف كيفية ارتباط الكيانات مع بعضها البعض. يمكن أن تكون العلاقات أحادية الاتجاه أو ثنائية الاتجاه.
3. السمات (Attributes): تمثل البيانات المحددة التي ترتبط بكل كيان. على سبيل المثال، إذا كانت الكيانات هي "العملاء"، فقد تحتوي السمات على اسم العميل، عنوانه، رقم هاتفه، وما إلى ذلك.



الشكل (1-4) مخطط ERD

1-2-4 جدول المستخدم :

جدول يحتوي على المعلومات الأساسية للعملاء ومديري النظام , وتم ربطه بجدول المدن لتحديد موقع المستخدم .

الجدول (1-4) جدول المستخدم

الحقل	الوصف	النوع	ملاحظات
id	معرف	bigint	مفتاح أساسي
name	اسم المستخدم	varchar	—
email	البريد الإلكتروني	varchar	فريد
password	كلمة المرور	varchar	—
phone	رقم الهاتف	varchar	—
role	نوع الحساب	varchar	—
fcm_token	رمز إشعارات الجهاز	text	—
city_id	معرف المدينة	bigint	مفتاح خارجي

2-2-4 جدول مزودي الخدمة (Providers) :

جدول يحتوي على البيانات الخاصة بالحرفيين ومزودي الخدمات، وتقييماتهم وخبراتهم.

الجدول (2-4) جدول مزودي الخدمة

الحقل	الوصف	النوع	ملاحظات
id	معرف	bigint	مفتاح أساسي
name	اسم مزود الخدمة	varchar	-
phone	رقم الهاتف	varchar	-
fcm_token	رمز إشعارات الجهاز	text	-
years_of_experience	سنوات الخبرة	int	-
status	حالة الحساب (نشط، محظور)	varchar	-
rating_avg	متوسط التقييم العام	decimal	-

3-2-4 جدول التصنيفات (Categories) :

جدول يحتوي على الأقسام والتصنيفات الرئيسية للخدمات المقدمة في النظام.

الجدول (3-4) جدول التصنيفات

الحقل	الوصف	النوع	ملاحظات
Id	معرف التصنيف	bigint	مفتاح أساسي
name	اسم التصنيف	varchar	-
slug	الرابط اللطيف للتصنيف (slug)	varchar	-
icon	أيقونة التصنيف	varchar	-

4-2-4 جدول الطلبات الخدمة (service_requests) :

جدول محوري يربط بين العميل، مزود الخدمة، والتصنيف، ويحتوي على تفاصيل وحالة الطلب.

الجدول (4-4) جدول الطلبات

الحقل	الوصف	النوع	ملاحظات
id	معرف الطلب	bigint	مفتاح أساسي

user_id	معرف العميل (طالب الخدمة)	bigint	مفتاح خارجي
provider_id	معرف المزود الخدمة	bigint	مفتاح خارجي
category_id	معرف التصنيف	bigint	مفتاح خارجي
address_id	معرف العنوان	bigint	مفتاح خارجي
description	تفاصيل الطلب	text	–
status	حالة الطلب (قيد الانتظار , مكتمل..)	varchar	–
Scheduled_at	موعد التنفيذ المجدول	timestamp	–

5-2-4 جدول ربط التصنيفات بمزودي الخدمة (category_provider) :

جدول وسيط (pivot Table) لحل العلاقة متعددة إلى متعدد (Many-to-Many), حيث يمكن لمزود الخدمة الانتماء لعدة تصنيفات, والتصنيف الواحد يضم عدة مزودين.

الجدول (5-4) جدول ربط التصنيفات بمزودي الخدمة

الحقل	الوصف	النوع	ملاحظات
category_id	معرف التصنيف	bigint	مفتاح أساسي, خارجي
provider_id	معرف المزود الخدمة	bigint	مفتاح أساسي, خارجي

6-2-4 جدول المفضلة (favorites) :

جدول يتيح للعملاء حفظ مزودي الخدمة في قائمة المفضلة للوصول السريع إليهم لاحقاً.

الجدول (6-4) جدول المفضلة

الحقل	الوصف	النوع	ملاحظات
id	معرف المفضلة	bigint	مفتاح أساسي
user_id	معرف العميل	bigint	مفتاح خارجي
provider_id	معرف مزود الخدمة	bigint	مفتاح خارجي

7-2-4 جدول المدفوعات (Payments) :

جدول لإدارة العمليات المالية وتسجيل السجلات الخاصة بكل طلب خدمة يتم دفعه.

الجدول (7-4) جدول المدفوعات

الحقل	الوصف	النوع	ملاحظات
id	معرف عملية الدفع	bigint	مفتاح أساسي
service_request_id	معرف الطلب الخدمة	bigint	مفتاح خارجي
user_id	معرف العميل	bigint	مفتاح خارجي
provider_id	طريقة مزود الخدمة	bigint	مفتاح خارجي
amount	المبلغ المدفوع	decimal	-
payment_method	طريقة الدفع (كاش إلكتروني...)	varchar	-
status	حالة الدفع (ناجح, قيد المعالجة..)	varchar	-

8-2-4 جدول التقييمات (Ratings) :

جدول مخصص لتخزين تقييمات وتعليقات العملاء حول مزودي الخدمة بعد إنجاز طلب الخدمة.

الجدول (8-4) جدول التقييمات

الحقل	الوصف	النوع	ملاحظات
id	معرف التقييم	bigint	مفتاح أساسي
user_id	معرف العميل	bigint	مفتاح خارجي
provider_id	معرف مزود الخدمة	bigint	مفتاح خارجي
service_request_id	معرف الطلب الخدمة المرتبط	bigint	مفتاح خارجي
rating	القيمة الرقمية للتقييم	integer	-
comment	نص التعليق أو الملاحظة	text	-

9-2-4 جدول المحادثات (conversations) :

جدول الأساسي لإنشاء الجلسة الخاصة بالمحادثة (غرفة الدردشة) بين أطراف النظام.

الجدول (9-4) جدول التقييمات

الحقل	الوصف	النوع	ملاحظات
id	معرف المحادثة	bigint	مفتاح أساسي
type	نوع المحادثة	varchar	–
service_request_id	معرف طلب الخدمة المرتبط (إن وجد)	bigint	مفتاح خارجي
status	حالة المحادثة	varchar	–

10-2-4 جدول المشاركين في المحادثة (conversation_participants) :

جدول وسيط يحدد الأطراف المشاركة في كل محادثة، يعتمد على علاقة Polymorohic لمرونة إضافة أي كيان للمحادثة (عميل، مزود خدمة، مدير).

الجدول (10-4) جدول المشاركين في المحادثة

الحقل	الوصف	النوع	ملاحظات
id	معرف السجل	bigint	مفتاح أساسي
conversation_id	معرف المحادثة	bigint	مفتاح خارجي
participant_type	نوع المشارك (Morph Type)	varchar	–
participant_id	معرف المشارك (Morph ID)	bigint	–

11-2-4 جدول الرسائل (Messages) :

جدول لتخزين تفاصيل الرسائل النصية المتبادلة داخل المحادثات.

الجدول (11-4) جدول الرسائل

الحقل	الوصف	النوع	ملاحظات
id	معرف الرسالة	bigint	مفتاح أساسي

مفتاح خارجي	bigint	معرف المحادثة	conversation_id
مفتاح خارجي	string bigint	معرف طلب المرتبط بالرسالة	service_request_id
–	varchar	نوع المرسل (Morph Type)	Sender_type
–	bigint	معرف المرسل (Morph ID)	sender_id
–	varchar	نوع الرسالة (نص , صورة ..)	type
–	text	محتوى الرسالة	body

12-2-4 جدول الإشعارات (Notifications) :

جدول مبني على نظام إشعارات إطار العمل الافتراضي (polymorphic) لتخزين كافة إشعارات النظام

الجدول (12-4) جدول الإشعارات

الحقل	الوصف	النوع	ملاحظات
id	معرف الإشعار	unid	مفتاح أساسي
notifiable	نوع الكيان المستقبل للإشعار	varchar	–
notifiable_id	معرف الكيان المستقبل	bigint	–
type	نوع الإشعار (فئة الإشعار)	varchar	–
data	بيانات الإشعار	json	–

13-2-4 جدول العناوين (addresses-Location Service) :

جدول يتبع لخدمة المواقع (Location Service) ويعتمد على العلاقات المتعددة (polymorphic) لربط العناوين بأي كيان في النظام.

الجدول (13-4) جدول الإشعارات المستخدم

الحقل	الوصف	النوع	ملاحظات
id	معرف العنوان	bigint	مفتاح خارجي
addressable_id	نوع المالك (Morph Type)	varchar	مفتاح خارجي
addressable_type	معرف المالك (Morph ID)	bigint	-
latitude	خط العرض	decimal(10,8)	-
longitude	خط الطول	decimal(11,8)	-
label	تسمية العنوان (منزل , عمل...)	varchar	-
Is_primary	هل العنوان الافتراضي؟	boolean	-
created_at	تاريخ ووقت الإنشاء	timestamp	-

14-2-4 جدول المدن (cities-Location Service) :

جدول يتبع لخدمة المواقع, يحتوي على قائمة المدن المتاحة لربطها بالمستخدمين وتسهيل إدارة العناوين.

الجدول (14-4) جدول المدن

الحقل	الوصف	النوع	ملاحظات
id	معرف المدينة	bigint	مفتاح أساسي
name	اسم المدينة (باللغة الإنكليزية)	varchar	-
name_ar	اسم المدينة (باللغة العربية)	varchar	-
created_at	تاريخ ووقت الإنشاء	timestamp	-

–	timestamp	تاريخ ووقت التعديل	updated_at
---	-----------	--------------------	------------

الفصل الخامس

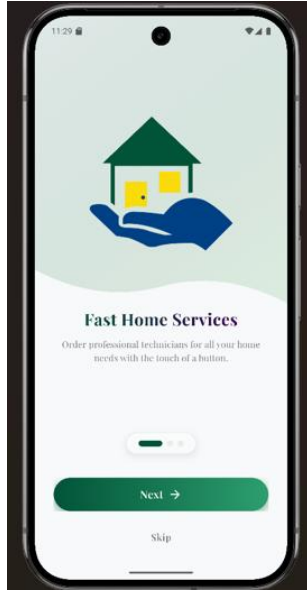
واجهات المشروع

5-1 مقدمة:

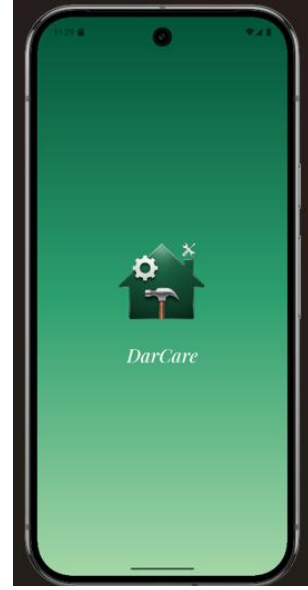
نستعرض بعض واجهات هذه التطبيق, نقسمها الى خمسة أقسام :

5-2 إدارة الحسابات والمصادقة (Authentication) :

أولاً: شاشات ترحيبية وتوجيهية.(Onboarding) كما في الشكل التالي .



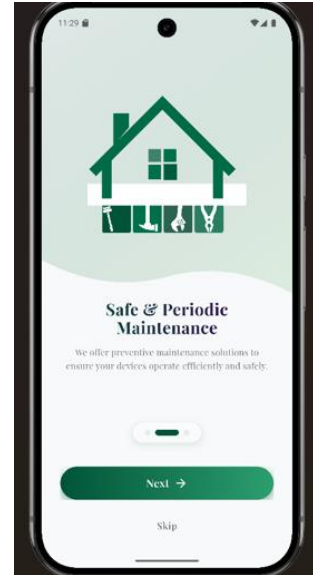
الشكل (2) : الشاشات التعريفية 1



الشكل (1) : شاشة البداية



الشكل (4) : الشاشات التعريفية 3



الشكل (3) : الشاشات التعريفية 2

ثانياً: نماذج تسجيل دخول وإنشاء حسابات مخصصة لكل دور (عميل / مقدم خدمة).

1. يعرض الشكل واجهات التسجيل المخصصة لكل دور في المنصة. تقتصر واجهة "العميل" على جمع البيانات الشخصية الأساسية (كالاسم، معلومات الاتصال، ومدينة الإقامة). في حين تتطلب واجهة "مقدم الخدمة" (الحرفي) بيانات إضافية مخصصة لبناء ملفه المهني، مثل تحديد التخصص، وسنوات الخبرة،

الشكل (7) : تسجيل مقدم الخدمة 2

الشكل (6) : تسجيل مقدم الخدمة 1

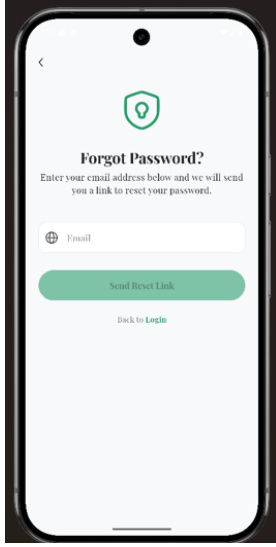
الشكل (5) : تسجيل الدخول

2. واجهة تحديد نوع الحساب (مستخدم أو حرفي)، عرض الشكل (8) و (9) واجهة اختيار الدور عند إنشاء حساب جديد، حيث تُوجّه المستخدم لاختيار واجهته المخصصة؛ إما كـ "عميل" للبحث عن الفنيين وطلب الخدمات المنزلية، أو كـ "مقدم خدمة/حرفي" لعرض مهاراته واستقبال الطلبات.

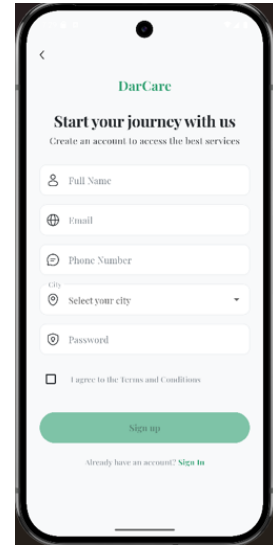
الشكل (9) : اختيار الدور 2

الشكل (8) : اختيار الدور 1

3. واجهات تسجيل الدخول وإعادة تعيين كلمة المرور (Login & Forgot Password Screens):
 ➤ يعرض الشكل واجهات المصادقة وأمان الحساب؛ حيث تُتيح واجهة تسجيل الدخول (Login) للتعامل الدخول إلى المنصة بإدخال البريد الإلكتروني وكلمة المرور مع خيار تذكّر البيانات، بينما تُمكن واجهة نسيان كلمة المرور (Forgot Password) المستخدم من إدخال بريده الإلكتروني لإرسال رابط آمن لإعادة ضبط كلمة المرور.



الشكل (11) : نسيان كلمة المرور

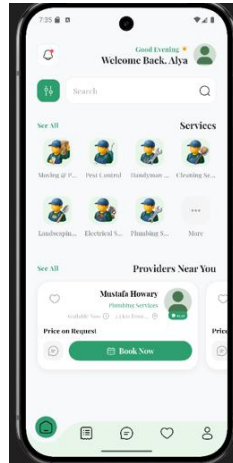


الشكل (10) : تسجيل المستخدم

3-5 واجهات العميل (User Module) :

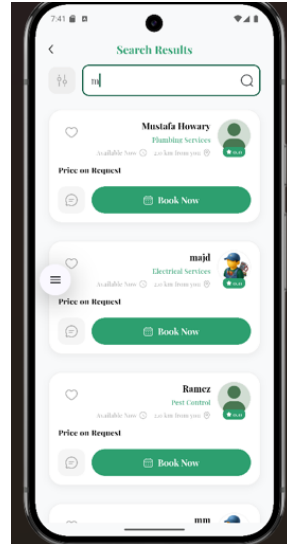
أولاً: الرئيسية و البحث :

1. يُظهر الشكل الشاشة الرئيسية للعميل، التي تتيح تصفح الخدمات المنزلية والبحث عنها وتصنيفتها حسب الفئة، مع عرض مزودي الخدمة القريبين ومعلومات التوافر والمسافة والتقييم وزر الحجز المباشر. كما توفر شريط تنقل للوصول إلى الطلبات والمحادثات والمفضلة والملف الشخصي.



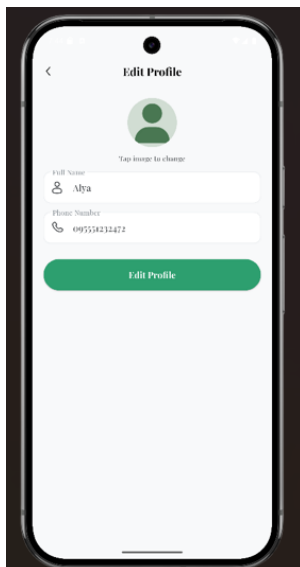
الشكل (12) : الصفحة الرئيسية للمستخدم

2. يعرض الشكل واجهة نتائج البحث التي تتيح للعميل استعراض الحرفيين المتاحين وفق البحث، مع إظهار التخصص، التقييم، المسافة، وحالة التوافر. كما توفر أزرارًا للحجز الفوري، المحادثة، وإضافة الحرفي إلى المفضلة.



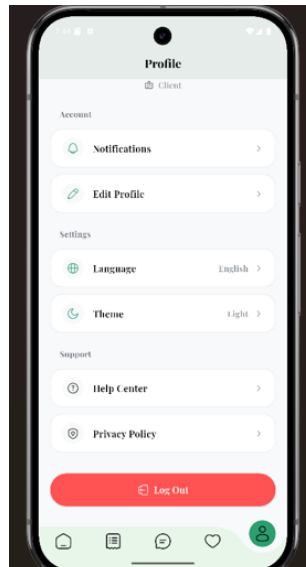
الشكل (13) : البحث

3. يعرض الشكل واجهات الملف الشخصي للعميل، التي تتيح إدارة بيانات الحساب وتخصيص تجربة الاستخدام، بما يشمل المعلومات الشخصية، الإشعارات، اللغة والمظهر، والدعم الفني. كما تتيح واجهة تعديل الملف تحديث البيانات ومزامنتها مع قاعدة بيانات النظام.



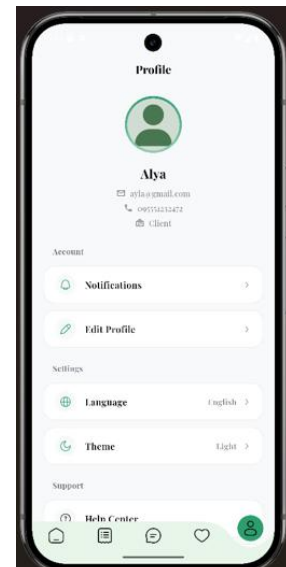
الشكل (16) :

تعديل الملف الشخصي للمستخدم



الشكل (15) :

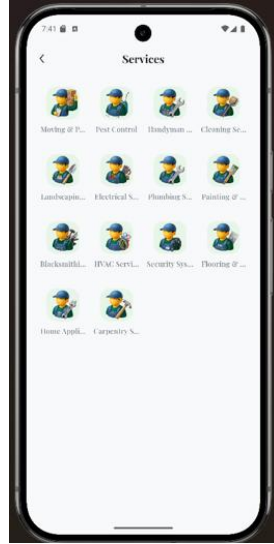
الملف الشخصي للمستخدم 2



الشكل (14) :

الملف الشخصي للمستخدم 1

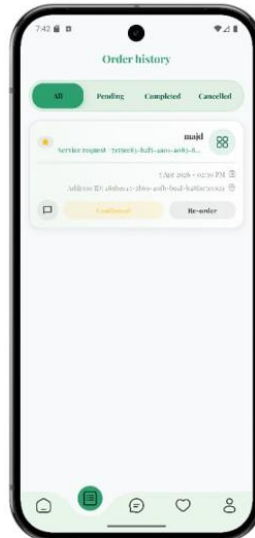
4. يعرض الشكل واجهة تصنيفات الخدمات في منصة DarCare، التي تتيح للعميل استعراض الأقسام واختيار الخدمة المطلوبة للانتقال إلى قائمة الحرفيين المتخصصين بها.



الشكل (17) : الخدمات

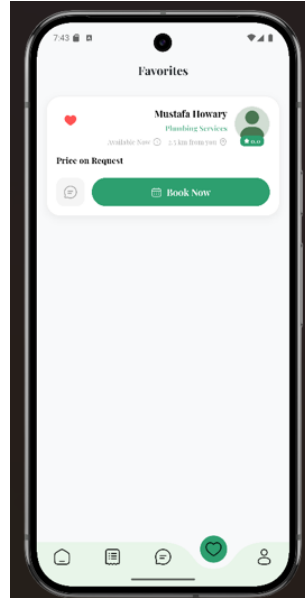
ثانياً: الطلبات و المفضلة :

1. يعرض الشكل واجهة سجل الطلبات، التي تتيح للعميل متابعة وإدارة طلباته حسب حالتها، مع عرض تفاصيل كل طلب ومقدم الخدمة والتاريخ. كما توفر الواجهة خيارات للتواصل مع الحرفي، تتبع الطلب، وإعادة جدولة الطلبات السابقة.



الشكل (18) : سجل الطلبات

2. يعرض الشكل واجهة قائمة المفضلة، التي تتيح للعميل الوصول السريع إلى مزودي الخدمة المحفوظين، مع عرض الاسم والتخصص والتقييم والتوافر والمسافة، بالإضافة إلى خيارات الحجز السريع والمحادثة المباشرة.

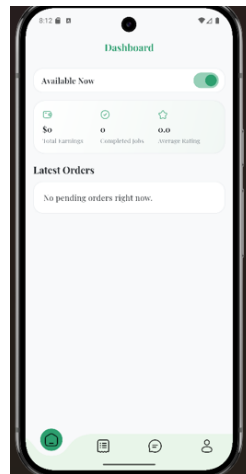


الشكل (19) : المفضلة

4-5 واجهات مقدم الخدمة/الحرفي (Provider Module) :

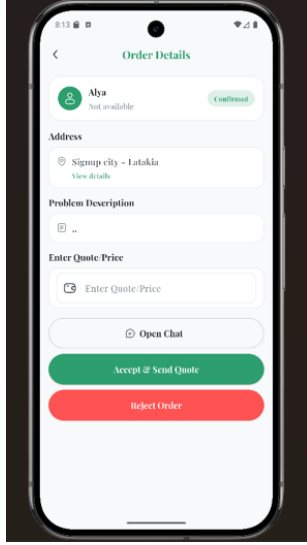
أولاً: لوحة التحكم

1. يعرض الشكل لوحة تحكم مزود الخدمة، التي تتيح إدارة التوافر واستقبال الطلبات، مع عرض مؤشرات الأداء مثل الأرباح والمهام المكتملة ومتوسط التقييم. كما تعرض أحدث الطلبات وتوفر تنقلاً سريعاً إلى الطلبات والمحادثات والملف الشخصي.

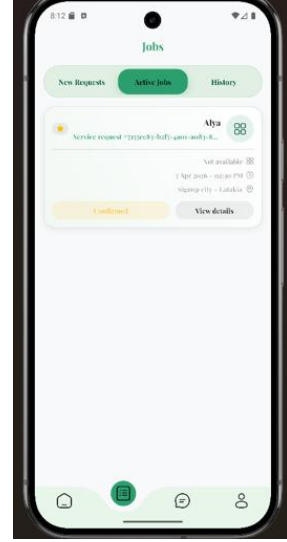


الشكل (20) : لوحة تحكم مقدم الخدمة

2. يعرض الشكل واجهات إدارة المهام لمزود الخدمة، التي تتيح تنظيم الطلبات إلى جديدة ونشطة ومنجزة. كما تعرض تفاصيل الطلب وموقع العميل ووصف المشكلة، مع خيارات التواصل، تقديم السعر، قبول أو رفض الطلب، وتحديث حالة الطلب عبر خدمة الطلبات.



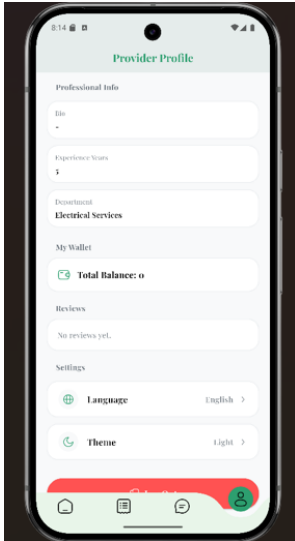
الشكل (22) : تفاصيل الطلب



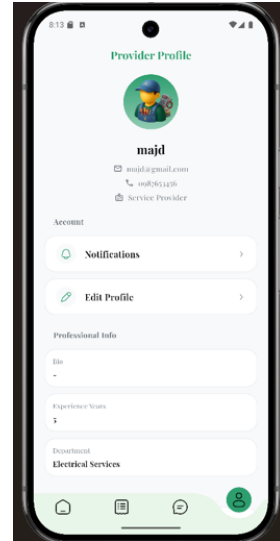
الشكل (21) : وظائف مقدم الخدمة

ثانياً: الملف المهني

1. يعرض الشكل واجهة الملف الشخصي لمزود الخدمة، التي تتضمن معلوماته الأساسية والمهنية مثل التخصص والخبرة والنبذة التعريفية. كما تتيح متابعة المحفظة والتقييمات، بالإضافة إلى إدارة إعدادات اللغة والمظهر.



الشكل (24) : الملف الشخصي لمقدم الخدمة 2



الشكل (23) : الملف الشخصي لمقدم الخدمة 1

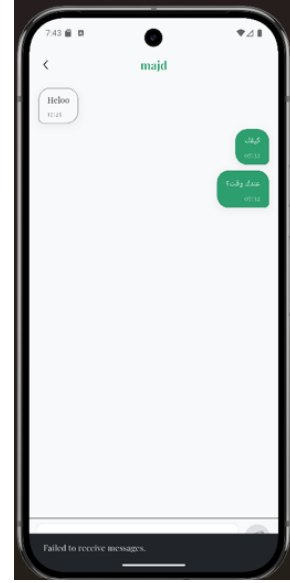
5-5 الخدمات المشتركة والتواصل (Shared Services) :

أولاً: المحادثات المباشرة والإشعارات: دردشة فورية بين العميل والحرفي عبر بروتوكول (WebSocket).

1. يعرض الشكل واجهتي المحادثات والدردشة المباشرة للعميل، حيث تتيح الأولى استعراض المحادثات مع مزودي الخدمة، بينما تمكن الثانية من التواصل المباشر لتنسيق تفاصيل الخدمة، مع دعم الاتصال اللحظي عبر WebSocket .

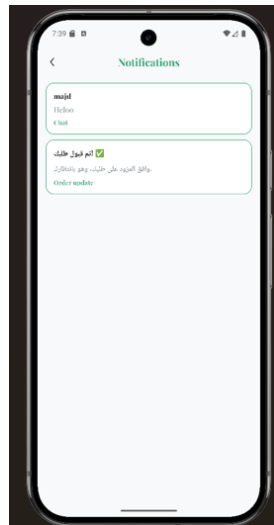


الشكل (26) : قائمة محادثات المستخدم



الشكل (25) : محادثة المستخدم

2. يعرض الشكل واجهة الإشعارات الخاصة بالعميل، التي تتيح متابعة التنبيهات اللحظية المتعلقة بالمحادثات وتحديثات حالة الطلبات، بما يضمن بقاء العميل على اطلاع مستمر بمستجدات خدماته .



الشكل (27) : إشعارات المستخدم

3. يعرض الشكل واجهتي المحادثات والدرشة المباشرة لمزود الخدمة، حيث تتيحان إدارة المحادثات مع العملاء والتواصل الفوري لمناقشة تفاصيل المهام والاستفسارات وعروض الأسعار، بما يضمن تنسيقاً فعالاً ومباشراً.



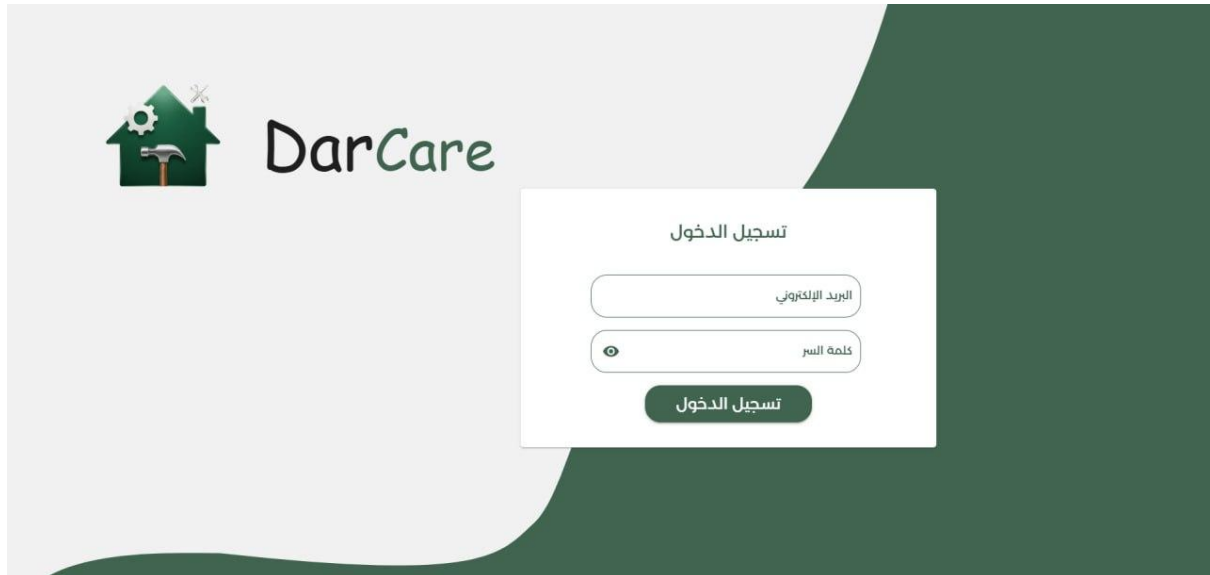
الشكل (29) : قائمة محادثات مقدم الخدمة



الشكل (28) : محادثة مقدم الخدمة

5-6 لوحة تحكم الأدمن (Admin Panel) :

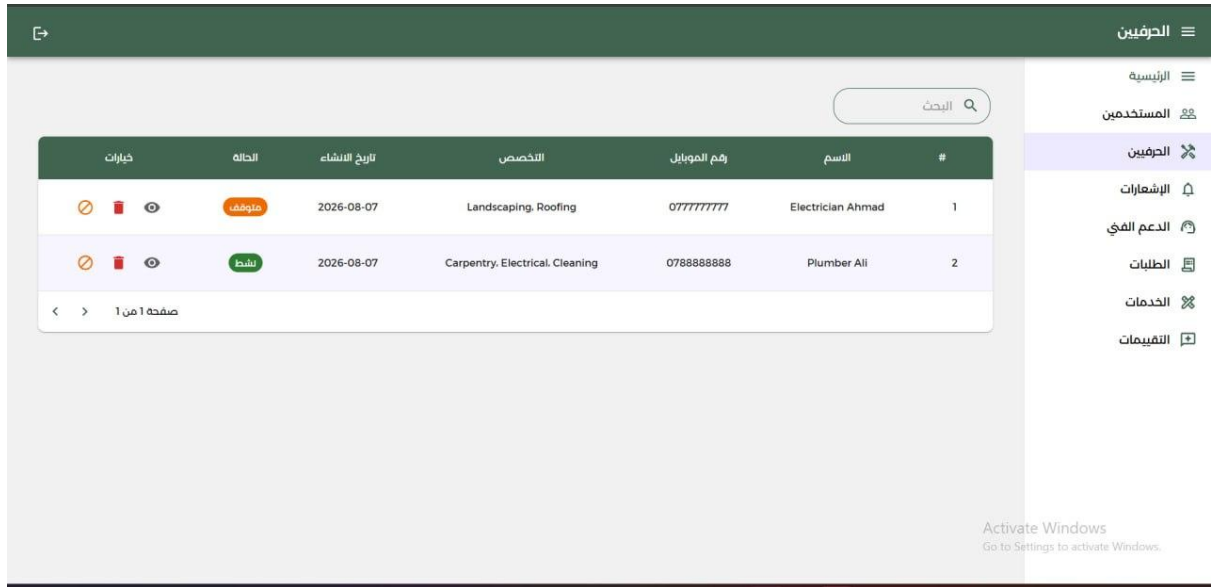
➤ إدارة مركزية عبر الويب لإحصائيات النظام، الحسابات، أقسام الخدمات، والتقييمات كما في الشكل التالي :



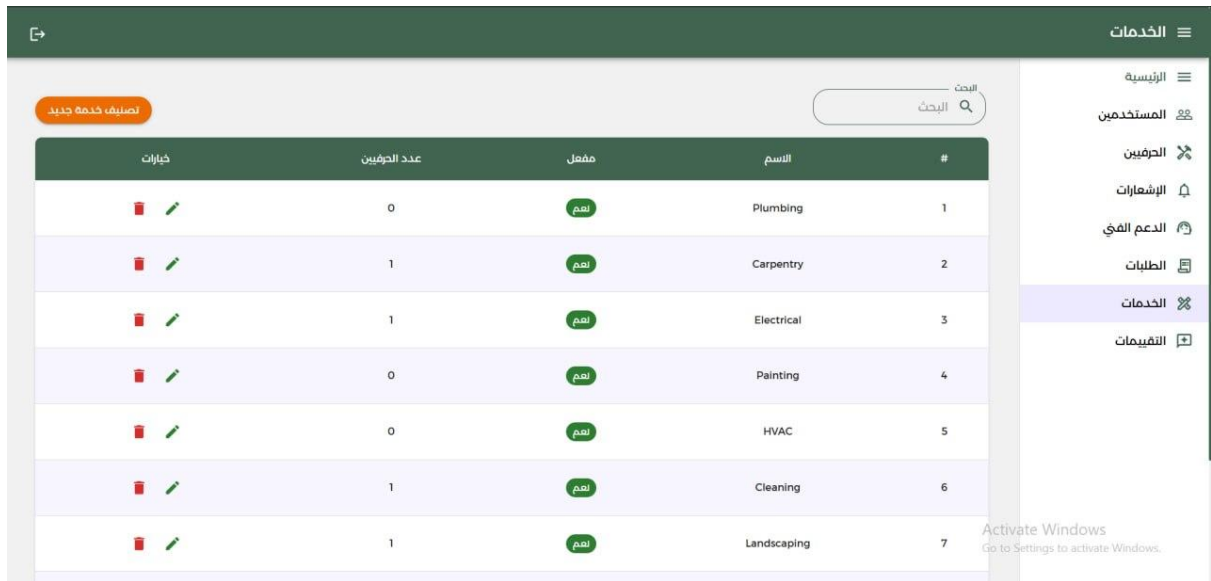
الشكل (30) : واجهة التسجيل الدخول لوحة التحكم



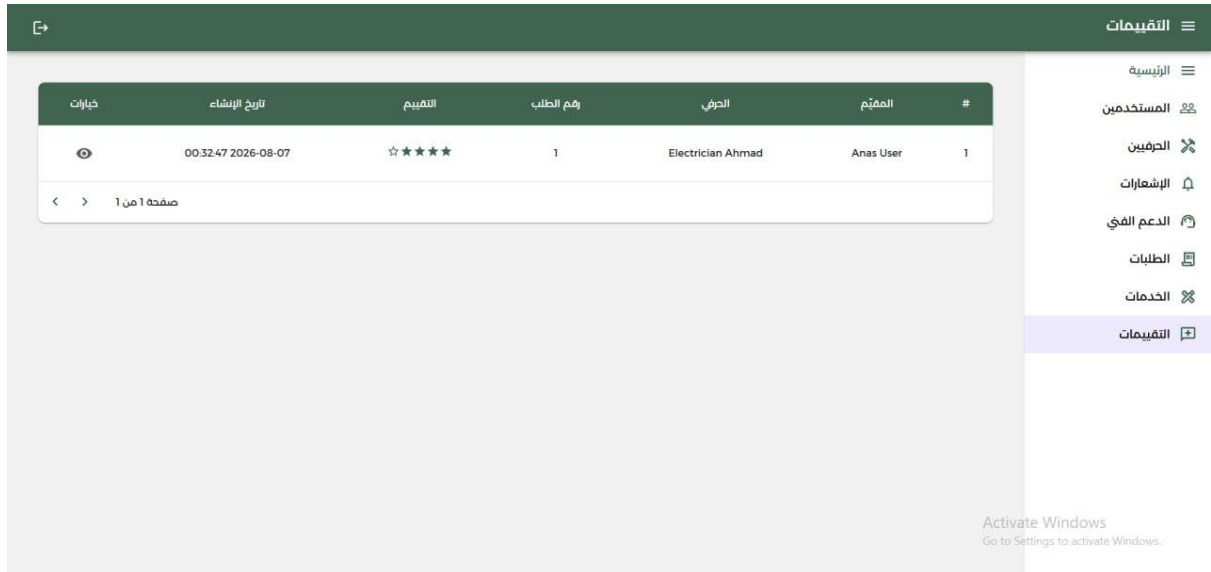
الشكل (31) : واجهة الرئيسية التي تعرض الإحصائيات والتقييمات وأحدث الطلبات



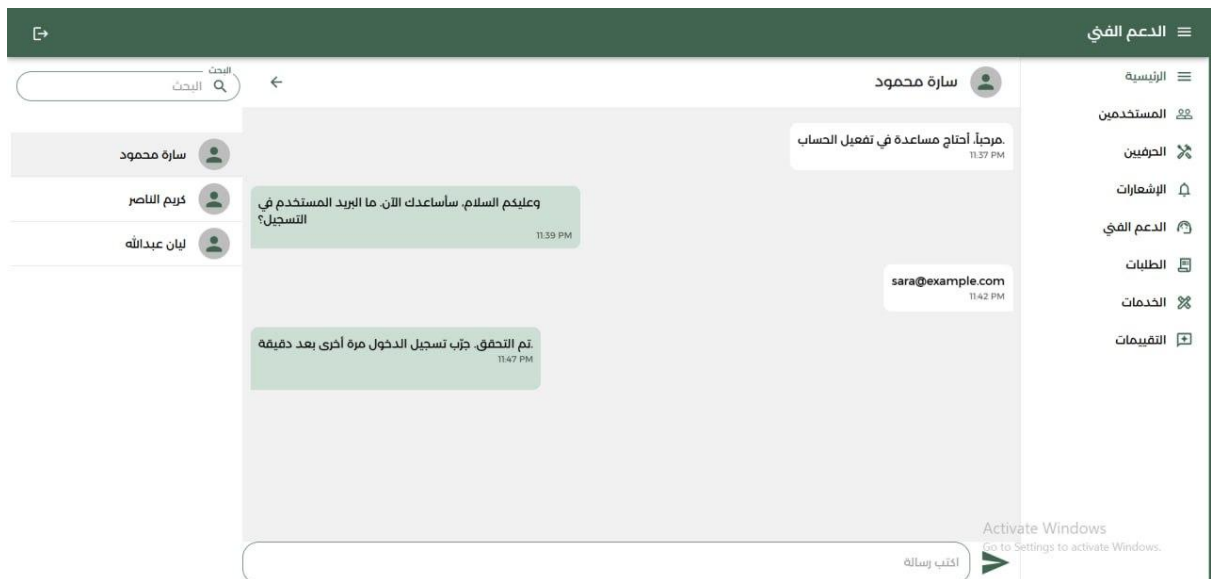
الشكل (32) : واجهة عرض الحرفيين النشطين في التطبيق



الشكل (33) : واجهة عرض الخدمات الموجودة في التطبيق



الشكل (34) : واجهة عرض تقييمات الحرفيين



الشكل (35) : واجهة الدعم الفني

الفصل السادس

الاستنتاجات والمقترحات

6-1 الاستنتاجات:

استنتجنا أن مشروع DarCare يوفر حلاً منظماً وسريعاً يربط العملاء بمقدمي خدمات الطوارئ المنزلية، ويسهم في تسهيل طلب الخدمة والوصول إلى الحرفي المناسب. كما ساعد اعتماد بنية معيارية (Modular Architecture) واستخدام التقنيات الحديثة في تحسين أداء النظام وتنظيم بياناته وتسريع وتيرة تطويره، إلى جانب توفير التواصل المباشر بين العميل ومقدم الخدمة. وبشكل عام، يمكن القول إن منصة DarCare مثّلت نموذجاً عملياً صالحاً لحل رقمي متكامل وقابل للتوسع مستقبلاً في مجال الخدمات المنزلية الطارئة، ساهم في تحويل العملية من أسلوب تقليدي غير منظم إلى نظام إلكتروني أكثر كفاءة وموثوقية، مع إمكانية تطويره لاحقاً ليشمل خدمات ومميزات أكثر تقدماً.

إثبات فاعلية البدء ببنية (Modular Monolith) كخطوة تمهيدية مدروسة للانتقال نحو (Microservices) ؛ حيث أثبت هذا النهج قدرته على تقليل المخاطر التقنية، وتسهيل عملية فصل الخدمات (Main و Location دون التسبب بانهيار النظام.

6-2 المقترحات:

في ضوء نتائج المشروع وإمكانياته الحالية، يمكن تطوير منصة DarCare مستقبلاً من خلال مجموعة من التحسينات التي تعزز من كفاءتها وموثوقيتها وتجربة المستخدم، ومن أبرزها:

1. تطوير نظام دفع إلكتروني آمن ومتكامل يدعم عدة وسائل دفع مع إمكانية إدارة الفواتير والاسترجاع.
2. تحسين نظام تحديد الموقع الجغرافي باستخدام تقنيات أكثر دقة، مع إمكانية تتبع مقدم الخدمة بشكل لحظي أثناء التنقل.
3. تعزيز نظام الإشعارات ليكون أكثر ذكاءً وتفاعلية، بحيث يواكب جميع مراحل الطلب بشكل فوري.
4. تطبيق آليات تحقق متقدمة لمقدمي الخدمات تشمل التوثيق، وتقييم الخبرة، وربط الحسابات بهويات رسمية لزيادة مستوى الأمان والثقة.
5. توسيع نطاق الخدمات لتشمل تخصصات إضافية تلبي احتياجات أكبر شريحة من المستخدمين.
6. تطوير لوحة التحكم الإدارية لتشمل تحليلات متقدمة (Analytics) تساعد في اتخاذ قرارات مبنية على البيانات مثل أداء مقدمي الخدمات ونسب الطلبات.
7. إمكانية إضافة نظام ذكاء اصطناعي لاحقاً لاقتراح أفضل مقدم خدمة بناءً على الموقع، التقييم، وسرعة الاستجابة.

3-6 التحديات :

واجه الفريق خلال التنفيذ عدة تحديات، أبرزها:

1. عدم استقرار الإنترنت والكهرباء، ما أثر على استمرارية العمل الجماعي عن بعد.
2. القيود الدولية على سوريا، التي حدّت من الوصول لبعض الخدمات السحابية وبوابات الدفع العالمية.
3. تعدد التقنيات المطلوب إتقانها Flutter ، React ، Laravel ، PostgreSQL ضمن وقت محدود.
4. غياب مستخدمين حقيقيين حال دون اختبار النظام ميدانياً وجمع ملاحظات فعلية.
5. تنسيق العمل بين ستة أعضاء موزعين على مهام مختلفة عبر Git.

➤ **التحديات المستقبلية : عند التوسع بالمنصة مستقبلاً، تبرز تحديات محتملة أهمها:**

1. دمج نظام دفع إلكتروني آمن، في ظل محدودية الخيارات المصرفية والقيود الدولية على سوريا.
2. ضمان استقرار الخدمات اللحظية الدردشة عبر WebSocket ، تتبع الموقع رغم تذبذب البنية التحتية.
3. بناء آليات توثيق موثوقة لهوية الحرفيين لضمان الأمان مع تزايد المستخدمين.
4. الانتقال التدريجي من Monolith إلى Microservices دون التأثير على استقرار النظام القائم.
5. تصميم نماذج تسعير وعمولات تواكب تقلب سعر الصرف والوضع الاقتصادي.
6. غياب إطار تشريعي واضح لتنظيم عمل منصات الخدمات المستقلة (Gig Economy) في سوريا حالياً.
7. مع زيادة عدد الخدمات المصغرة وتوسع المنصة مستقبلاً، يُقترح الانتقال من نمط الاتصال المباشر (Point-to-Point) إلى تطبيق بوابة مركزية للخدمات (API Gateway) لتنظيم حركة المرور (Traffic Routing) ، مركزية التحقق من الهوية، وتسهيل اكتشاف الخدمات (Service Discovery) .

1. **CSS Docs**
[CSS: Cascading Style Sheets | MDN \(mozilla.org\).](#)
2. **Dart Documentation**
[Dart programming language | Dart.](#)
3. **Flutter Documentation**
[Flutter - Build apps for any screen.](#)
4. **Git Documentaiton**
[Git - Documentation \(git-scm.com\).](#)
5. **HTML Docs**
[HTML: HyperText Markup Language | MDN \(mozilla.org\).](#)
6. **JavaScript Docs**
[JavaScript documentation — DevDocs.](#)
7. **MySQL**
[Oracle. \(2026\). MySQL 8.4 Reference Manual. MySQL Documentation.](#)
8. **React**
[React Homepage.](#)
9. **TypeScript Docs**
[TypeScript: Documentation - TypeScript for JavaScriptProgrammers](#)
10. **Laraval**
[Laravel Official Documentation.](#)
11. **Modular Monolith Architecture**
[Modular **Monolith: A Primer** .](#)
12. **Microsoft Modular Architecture Guide**
[Modular MonolithArchitecture \(Microsoft eShop\).](#)
13. **Martin Fowler - Monolith First**
[MonolithFirst\(martinfowler.com\).](#)
14. **Software Modularity Concepts**
[Modular Programming\(GeeksforGeeks\) .](#)
15. **Lewis, J., & Fowler, M. (2014). *Microservices: A Definition of This New Architectural Term*. Martin Fowler.**

شكر وتقدير

نود في الاختتام أن نتوجه بجزيل الشكر لجميع الأساتذة والدكاترة في كليتنا العزيزة كلية الهندسة المعلوماتية على جهودهم المبذولة في تعليمنا وإرشادنا وتوجيهنا. ونود أيضا أن نتقدم بالشكر الجزيل لجميع المهندسين والمهندسات على مساعدتهم الدائمة لنا. أخيرا نود أن نتقدم بالشكر الجزيل للدكتورة مروة دحدوح على هذا المشروع والذي كان عوننا لنا في مختلف المشاكل التي واجهتنا. وشكرا...

Abstract:

The project aims to develop an online platform that connects individuals with home emergency maintenance service providers (such as electricity, plumbing, air conditioning, etc.) in a fast and reliable manner. This idea addresses the common challenge of finding trustworthy technicians quickly, especially in urgent situations, where people often rely on unverified sources like social media, exposing them to delays, poor service quality, or potential fraud. The system is designed as a unified application with role-based access, allowing users to choose their role (customer or technician) within the same mobile application, in addition to a dedicated administrative dashboard.

1. Mobile Application (User & Technician) : A single mobile application developed using the Flutter framework. It allows individuals to register and select their role as either a user or a technician.

As a user, individuals can request emergency services by providing details about the issue and location, receive multiple offers, communicate with technicians, and select the most suitable option based on price and ratings.

As a technician, users can create professional profiles, showcase their skills, browse nearby service requests, apply for jobs, communicate with clients, and complete tasks while building their reputation through ratings.

2. Admin Dashboard: A web-based dashboard developed using the React framework. It provides administrators with full control over the platform, including managing users and technicians, verifying accounts, monitoring service requests, and ensuring the quality, security, and reliability of the system.

Overall, the platform offers an integrated and efficient solution that enhances accessibility to home emergency services, reduces risks, and improves user experience for both customers and service providers.

University of Aleppo
Faculty of Informatics Engineering
Department of Software Engineering
And Information System



Emergency Home Services Management Application

DarCare

BY

Amr Kallas

Aya Hamoud

Mahdeya Mayati

Enas Balouch

Laila Haddad

Mohammad Hanifeh Al-Ahmad

Supervised By

Dr. Marwa Dahdouh

2026-2025

Submitted in partial fulfillment of the requirements of degree of Bachelor in
the Faculty of Informatics Engineering